

Efficient Parallel Reduction on GPUs with Hipacc

Bo Qiao[‡], Oliver Reiche[†], M. Akif Özkan[‡], Jürgen Teich[‡], Frank Hannig[‡]
 {bo.qiao, akif.oezkan, juergen.teich, frank.hannig}@fau.de, oliver.reiche@siemens-healthineers.com

[‡]Friedrich-Alexander University Erlangen-Nürnberg (FAU), Germany

[†]Siemens Healthineers AG, Erlangen, Germany

ABSTRACT

Hipacc is a domain-specific language for ease of programming image processing applications on hardware accelerators such as GPUs. It relieves the burden of manually porting algorithms to hardware for developers with the help of domain- and architecture-specific knowledge. One fundamental operation in image processing is reduction. Global reduction operators are the building blocks of many widely used algorithms, including image normalization, similarity estimation, etc. This paper presents an efficient approach to perform parallel reductions on GPUs with Hipacc. Our proposed approach benefits from the continuous effort of performance and programmability improvement by hardware vendors, for example, by utilizing the latest low-level primitives from Nvidia. Results show our approach achieves a speedup of up to 3.43 over an existing Hipacc implementation with traditional optimization methods, and a speedup of up to 9.02 over an implementation using the Thrust library from Nvidia.

CCS CONCEPTS

• **Computing methodologies** → **Image processing; Graphics processors**; • **Software and its engineering** → **Domain specific languages**;

KEYWORDS

Domain-Specific Languages, Parallel Reduction, GPUs

ACM Reference Format:

Bo Qiao, Oliver Reiche, M. Akif Özkan, Jürgen Teich, and Frank Hannig. 2020. Efficient Parallel Reduction on GPUs with Hipacc. In *23rd International Workshop on Software and Compilers for Embedded Systems (SCOPES '20)*, May 25–26, 2020, Sankt Goar, Germany. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3378678.3391885>

1 INTRODUCTION

Porting image processing algorithms to hardware accelerators such as graphics processing units (GPUs) has long been a tedious task and can only be performed by a small group of architecture experts. Despite the introduction of frameworks such as CUDA and OpenCL, achieving high-performance implementations remains

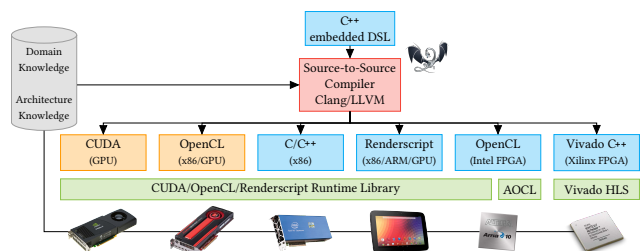


Figure 1: Overview of the Hipacc framework and its target architectures. GPU targets highlighted in orange.

challenging and often suffers from portability. Code tuned for one architecture is not guaranteed to be efficient on another architecture. As a remedy, domain-specific languages (DSLs) have emerged for ease of programming applications within a certain domain. It relieves the burden of manually porting algorithms to hardware for developers with the help of domain- and architecture-specific knowledge. Well-known image processing DSLs such as Halide [1], PolyMage [2], and Hipacc [3] provide a very concise way of describing algorithms and hide the implementation details inside the compiler, thus achieving a good compromise between performance and functional portability. Application development with DSLs is based on abstractions. Common compute patterns within a specific domain can be exploited and provided to the user through language constructs. In image processing, algorithms can be categorized based on *what information* is used to map one image to another [4]. Here, the three primary classes are *pixel operators*, computing an output pixel based on a single input pixel, *local operators*, additionally incorporating the local neighborhood within the input image, and *operators with multiple images*, where several input images are used. A DSL such as Hipacc follows this approach and classifies the algorithms as *point*, *local*, and *global* operators.

In this paper, we focus on optimizing reductions, a global operator in Hipacc, and a fundamental, critical building block of many widely used image processing applications. For example, detecting the global minima or maxima for image normalization, or the mean value for similarity estimation. Due to the importance of reduction operations, efficient implementation techniques on targets such as GPUs have been studied extensively [5, 6]. Despite the continuous effort in algorithm innovation and scheduling, the rapid evolution of architectures with better programmability and performance support enables new opportunities for optimization. For example, Nvidia not only introduced new low-level intrinsics such as the shuffle instructions [7] but also enhanced the performance of existing APIs such as the atomic functions [8]. Those new features have drawn attention in several recent works [9, 10]. Dakkak et al. utilize the state-of-the-art Tensor Cores on Nvidia GPUs for performing

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SCOPES '20, May 25–26, 2020, Sankt Goar, Germany

© 2020 Association for Computing Machinery.

ACM ISBN 978-1-4503-7131-5/20/05...\$15.00

<https://doi.org/10.1145/3378678.3391885>

warp-level reductions using shuffle instructions [9]. De Gonzalo et al. optimize GPU code generation in a high-level programming framework called Tangram, with the help of atomic functions as well as shuffle instructions [10]. Currently, Hipacc generates very efficient implementations for CUDA-enabled GPUs using traditional optimization techniques. Nevertheless, the existing implementation is not yet able to benefit from the mentioned new features in CUDA. We propose a new approach that takes advantage of those features in combination with the architecture-specific knowledge in Hipacc. Our contributions are as follows:

- (1) An efficient parallel reduction implementation by extending the traditional optimization techniques with new features in CUDA, such as the shuffle instructions.
- (2) A compiler-based tool flow in Hipacc that switches optimizations automatically based on architecture knowledge.

The remainder of this paper is organized as follows: Section 2 introduces the language construct for implementing reduction kernels in the Hipacc DSL. Section 3 presents the traditional optimization methods as well as the proposed new approach in Hipacc. The results are provided in Section 4 before we conclude in Section 5.

2 THE HIPACC FRAMEWORK

The Hipacc framework consists of an open-source¹ image processing DSL embedded into C++ and a source-to-source compiler. It supports code generation for a variety of backend targets ranging from multicore CPUs, GPUs to FPGAs. An overview of the framework and its supported architectures is depicted in Figure 1.

2.1 Source-to-source Compilation

In contrast to some DSLs such as Halide [1] and PolyMage [2] that directly emit target code, Hipacc performs source-to-source compilation. The input DSL code is parsed using Clang to generate the abstract syntax tree (AST). Then, the AST representation is used as an intermediate representation in Hipacc to perform domain-specific optimizations such as kernel fusion or concurrent kernel execution [11, 12, 13]. Finally, the generated code is pretty-printed from the transformed AST.

2.2 Global Reduction Kernel

Reductions in Hipacc can be specified by implementing a user-defined class. This class should be derived from the *Kernel* class in the framework, which provides a virtual *reduce()* function to be implemented in the user class. An example is depicted in Listing 1. The *reduce()* method takes two pixels as inputs and describes the operation applied to the entire input image. The final result can be retrieved using the *reduce_data()* method of the base class. In this

```
#include "hipacc.hpp"
class MaxReduction : public Kernel<int> {
// ...
int reduce(int left, int right) const {
    return max(left, right);
}
};
```

Listing 1: Reduction max in Hipacc

¹<http://hipacc-lang.org>

paper, we focus on the optimization of simple use cases, where 2D images are linearized into 1D for global reduction.

3 PARALLEL REDUCTION ON GPUS

The traditional GPU approach to compute a reduction in parallel can be summarized in three steps. First, data is loaded from the global memory: The operations are executed in the granularity of threadblocks, each block reads the data into their shared memory. Then, within each threadblock (intra-block), a tree-based approach is used to compute the partial result for this particular block. Finally, the partial results across all the blocks (inter-block) are collected, before the same approach is applied to obtain the final result. Figure 2 depicts an overview of the steps. Next, we discuss the traditional approach as well as the proposed optimizations for each step.

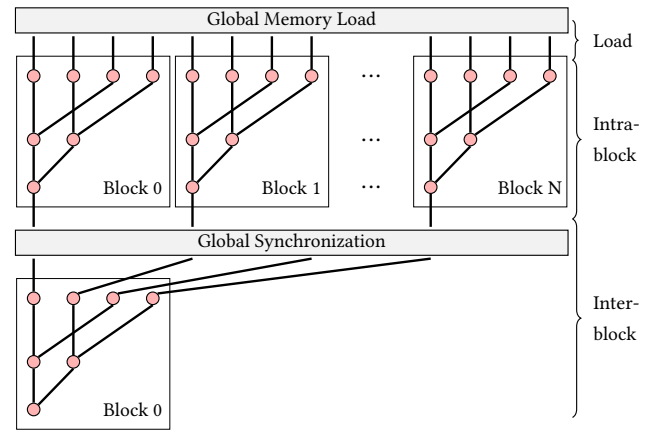


Figure 2: Parallel Reduction on GPUs

3.1 Global Memory Load

Reductions are known to have low arithmetic intensity: Roughly one floating-point operation per load [14]. Therefore it is critical to improve the achieved bandwidth during global memory access. Hipacc currently employs two methods: *Grid-stride loops* to optimize for coalesced memory access [15]. By executing inside a for-loop with an incremental size of the entire grid, the memory load achieves unit-stride within each warp and can handle arbitrary large input size. *Reduce during load* is the second method that optimizes the number of blocks during the memory load [14]. Because during reduction, half of the threads in each iteration are wasted, minimizing the number of executed threadblocks improves the achieved bandwidth.

3.1.1 Vectorized Load. We propose to use *vectorized load* [16] for further optimization. Vector data types such as int2 and float4 are provided in the CUDA headers, for which NVCC will generate vectorized load instructions in the assembly code. For example, LD.E.64 for int2 instead of LD.E for int, which implies the total number of executed instructions is halved. Additionally, vectorized load can be combined with the two existing methods without extra cost. After loading each vector data, threads can unpack the pixels within their own register space and perform a first reduction. An example is depicted in Figure 3.

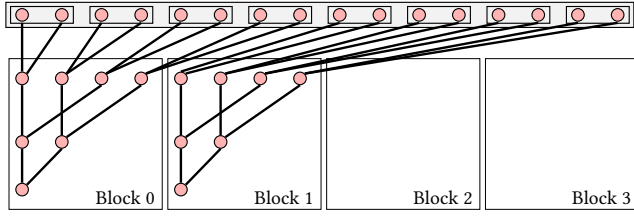


Figure 3: Global memory load with a vector size 2

3.2 Intra-block Reduce

After the global memory load, each threadblock continues to perform reductions within its own shared memory. Hipacc currently employs two methods to optimize the usage of shared memory: *Sequential addressing* is used to avoid bank conflict [14]. Instead of reading two consecutive addresses for reduction (default indexing), each thread in the block follows a rank-based indexing to avoid the 2-way bank conflict. Read and write to the shared memory must be synchronized within each block, which is a costly operation. Nevertheless, when the number of working threads is smaller than the warp size, the synchronization is not needed anymore due to the SIMT execution among warps. *Warp unrolling* [14] is used to unroll the rest of the loop in this situation.

3.2.1 Shuffle Instruction. The traditional approach to exchange data within a threadblock is via shared memory. However, for Kepler and later architectures, Nvidia introduced new low-level primitives called shuffle instructions [17]. The new instructions enable threads to exchange data within a warp without going through the shared memory, which avoid costly operations such as synchronization and generate smaller memory footprint. We propose to execute the intra-block reductions using shuffle instructions. The implementation of a warp reduction is given in Listing 2.

```
// reduce within a warp
__device__ inline int reduce_warp_max(int val)
{
    val = max(val, __shfl_down_sync(FULL_MASK, val, 16));
    val = max(val, __shfl_down_sync(FULL_MASK, val, 8));
    val = max(val, __shfl_down_sync(FULL_MASK, val, 4));
    val = max(val, __shfl_down_sync(FULL_MASK, val, 2));
    val = max(val, __shfl_down_sync(FULL_MASK, val, 1));
    return val;
}
```

Listing 2: Warp reduce with shuffle instructions

After the warp reduction, the partial results are written back to the shared memory. The expensive part has already been done by the shuffle instructions. At this point, only a few elements (4 for block size 128) need to be further reduced. This is performed by threads within the first warp, which requires no synchronization.

3.3 Inter-block Reduce

To make sure all threadblocks finish their executions, a global synchronization must be observed. The traditional solution for this problem is to use a second kernel, which waits for all blocks from the previous kernel to finish before execution starts. Currently, Hipacc employs a *single-pass kernel* approach that uses the `_threadfence()` intrinsic and a global `atomicInc()` function to produce a

single value in a single kernel. This is achieved by the first thread of each block updating a global memory counter using the atomic function. The last finished block is responsible for collecting the partial results and executes the final reduction.

3.3.1 Atomic Function. We propose to exploit the performance of atomic functions on new Nvidia architectures. CUDA offers arithmetic functions such as add, sub, and max, as well as bitwise functions such as and, xor. More generic operations can be implemented using the compare-and-swap function `atomicCAS()`. In this work, we focus on simple operations such as max and sum. Utilizing atomic functions requires less memory footprint, and the performance is scalable for larger data size. However, implementations running on not the newest architectures are known to be inefficient. Hipacc addresses this issue with its architecture knowledge by automatically fall back to the single-pass kernel approach.

Finally, Table 1 depicts a summary of all the discussed methods.

Optimizations	Baseline	Proposed	Require
Grid-stride loops	✓	✓	-
Reduce during load	✓	✓	-
Vectorized load		✓	-
Sequential addressing	✓	✓	-
Warp unrolling	✓		-
Shuffle instruction		✓	> Kepler
Single-pass kernel	✓		> CUDA 2.2
Atomic function		✓	> Kepler

Table 1: Baseline and proposed Hipacc implementations

4 EVALUATION AND RESULTS

We benchmarked three basic reduction operations: min, max, and sum, on two Nvidia GPUs: K20c with Kepler architecture and RTX2080 with Turing architecture. Each operation is applied to 1D images ranging from 1K (2^{10}) to 4M (2^{24}) pixels. We compare among three implementations: The baseline (current Hipacc) implementation, the proposed Hipacc implementation, and an implementation using Thrust [18], the high-level parallel algorithm library from Nvidia. Two speedups are computed and depicted in Figure 4: The speedup of the proposed approach over the baseline implementation, and the speedup of the proposed approach over the Thrust implementation. Execution time is measured as the median of 20 executions each.

4.1 Discussion

Compared to the baseline implementation, our proposed approach achieves a consistent speedup from 1.10 to 3.34 across two different architectures. One is Kepler and the other is Turing. It can be observed in Figure 4 that when the reduction images are above a certain size, the achieved speedups increase along with the image size. For small images, the achieved speedups show no increase due to the device resource availability. For example, the RTX2080 has sufficient resources to compute up to 8 threadblocks concurrently per SM for the min reduction kernel. This implies images smaller than 2^{16} do not have enough data to occupy the GPU fully. The speedups observed for small images mainly come from the use of vectorized load and atomic functions. The improvement by using shuffle instructions can be observed on large images.

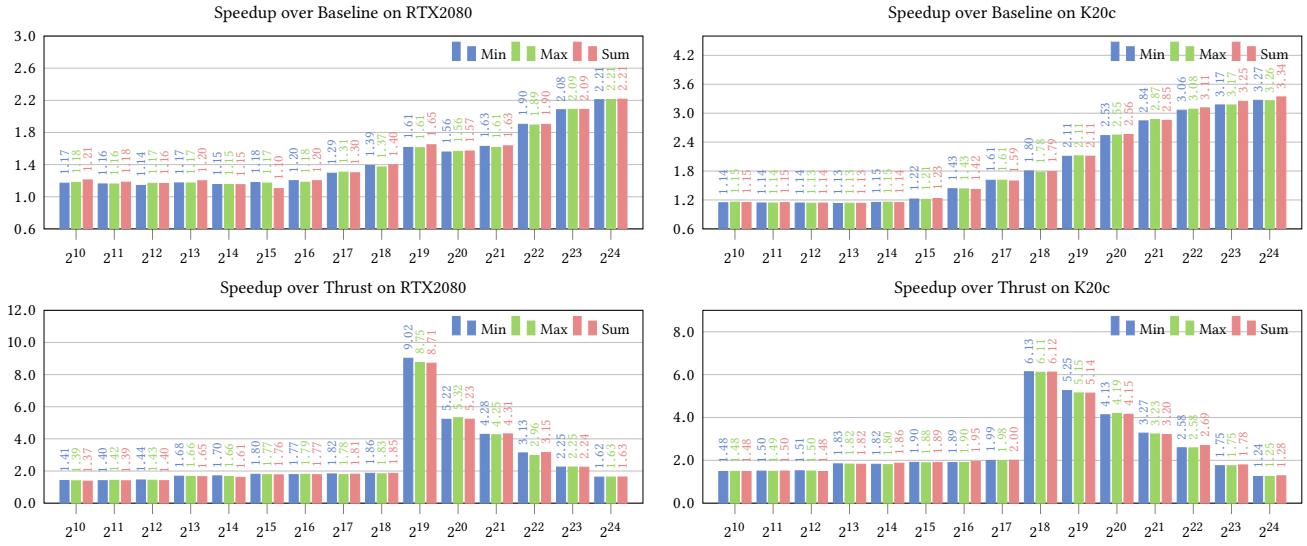


Figure 4: Various speedups plotted against the y-axis, and the number of pixels to be reduced is plotted against the x-axis

Compared to the Thrust implementation, our approach also achieves a consistent speedup from 1.37 to 9.02. After profiling, we observed that the performance difference is mainly due to the generation of kernel launch configurations. Hipacc employs its default configuration (128 block size) for the benchmarked architectures. While Thrust uses its internal tuner to decide on the launch configurations. For example, on the RTX2080, Thrust uses a constant block size of 256 for all the images with different grid sizes (up to 920). Moreover, Thrust executes two kernels for each reduction. Exploring the optimal configuration is planned for future work.

5 CONCLUSION

We improved the performance of the reduction implementation in Hipacc, a state-of-the-art image processing DSL and source-to-source compiler. Thanks for the enhancement of function support and the availability of new low-level intrinsics from Nvidia, high-level frameworks such as Hipacc can greatly benefit from the cutting edge techniques. As a next step, we plan to explore the performance impact of kernel launch configurations as well as the use of compare-and-swap functions for generic reduction kernels.

Acknowledgments: This work has been partially supported by Siemens Healthineers AG, Erlangen, Germany.

REFERENCES

- [1] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe. "Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines". In: *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)* (Seattle, WA, USA). ACM, 2013, pp. 519–530. doi: 10.1145/2491956.2462176.
- [2] R. T. Mullapudi, V. Vasista, and U. Bondhugula. "PolyMage: Automatic Optimization for Image Processing Pipelines". In: *ACM SIGARCH Computer Architecture News* 43.1 (Mar. 2015), pp. 429–443. doi: 10.1145/2786763.2694364.
- [3] R. Membarth, O. Reiche, F. Hannig, J. Teich, M. Körner, and W. Eckert. "HIPAcc: A Domain-Specific Language and Compiler for Image Processing". In: *IEEE Transactions on Parallel and Distributed Systems* (2016). doi: 10.1109/TPDS.2015.2394802.
- [4] I. N. Bankman. *Handbook of Medical Image Processing and Analysis*. Vol. 2. Academic Press, Dec. 2008. ISBN: 978-0-123-73904-9.
- [5] P. J. Martin, L. F. Ayuso, R. Torres, and A. Gavilanes. "Algorithmic strategies for optimizing the parallel reduction primitive in CUDA". In: *Proceedings of the International Conference on High Performance Computing Simulation (HPCS)*. 2012, pp. 511–519. doi: 10.1109/HPCS.2012.6266966.
- [6] S. Sengupta, M. Harris, and M. Garland. *Efficient parallel scan algorithms for GPUs*. Tech. rep. 2008.
- [7] J. Demouth. *Shuffle: Tips and Tricks*. 2013. URL: <http://on-demand.gputechconf.com/gtc/2013/presentations/S3174-Kepler-Shuffle-Tips-Tricks.pdf>.
- [8] S. J. Lars Nyland. *Understanding and Using Atomic Memory Operations*. 2013. URL: <http://on-demand.gputechconf.com/gtc/2013/presentations/S3101-Atomic-Memory-Operations.pdf>.
- [9] A. Dakkak, C. Li, J. Xiong, I. Gelado, and W.-m. Hwu. "Accelerating Reduction and Scan Using Tensor Core Units". In: *Proceedings of the ACM International Conference on Supercomputing (ICS)* (Phoenix, AZ, USA). ACM, 2019, pp. 46–57. doi: 10.1145/3330345.3331057.
- [10] S. G. De Gonzalo, S. Huang, J. Gómez-Luna, S. Hammond, O. Mutlu, and W.-m. Hwu. "Automatic Generation of Warp-Level Primitives and Atomic Instructions for Fast and Portable Parallel Reduction on GPUs". In: *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization (CGO)* (Washington, DC, USA). IEEE Press, 2019, pp. 73–84. doi: 10.1109/CGO.2019.8661187.
- [11] B. Qiao, O. Reiche, F. Hannig, and J. Teich. "Automatic Kernel Fusion for Image Processing DSLs". In: *Proceedings of the 21st International Workshop on Software and Compilers for Embedded Systems (SCOPES)* (Sankt Goar, Germany). ACM, 2018, pp. 76–85. doi: 10.1145/3207719.3207723.
- [12] B. Qiao, O. Reiche, F. Hannig, and J. Teich. "From Loop Fusion to Kernel Fusion: A Domain-Specific Approach to Locality Optimization". In: *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization (CGO)* (Washington, DC, USA). IEEE Press, 2019, pp. 10.1109/CGO.2019.8661176.
- [13] B. Qiao, O. Reiche, J. Teich, and F. Hannig. "Unveiling Kernel Concurrency in Multiresolution Filters on GPUs with an Image Processing DSL". In: *Proceedings of the 13th Annual Workshop on General Purpose Processing Using Graphics Processing Unit (GPGPU)* (San Diego, CA, USA). ACM, 2020, pp. 11–20. doi: 10.1145/3366428.3380773.
- [14] M. Harris. *Optimizing Parallel Reduction in CUDA*. 2007. URL: <https://developer.download.nvidia.com/assets/cuda/files/reduction.pdf>.
- [15] M. Harris. *Write Flexible Kernels with Grid-Stride Loops*. 2013. URL: <https://devblogs.nvidia.com/cuda-pro-tip-write-flexible-kernels-grid-stride-loops/>.
- [16] J. Luitjens. *Increase Performance with Vectorized Memory Access*. 2013. URL: <https://devblogs.nvidia.com/cuda-pro-tip-increase-performance-with-vectorized-memory-access/>.
- [17] J. Luitjens. *Faster Parallel Reductions on Kepler*. 2014. URL: <https://devblogs.nvidia.com/faster-parallel-reductions-kepler/>.
- [18] N. Bell and J. Hoberock. "Thrust: A Productivity-Oriented Library for CUDA". In: *GPU Computing Gems Jade Edition*. Applications of GPU Computing Series. Morgan Kaufmann, 2012. doi: 10.1016/B978-0-12-385963-1.00026-5.